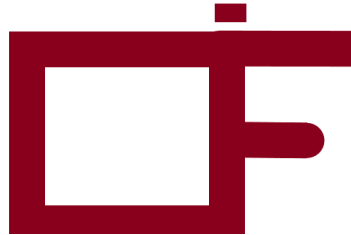


Regression

Machine Learning

TG1 Fernando

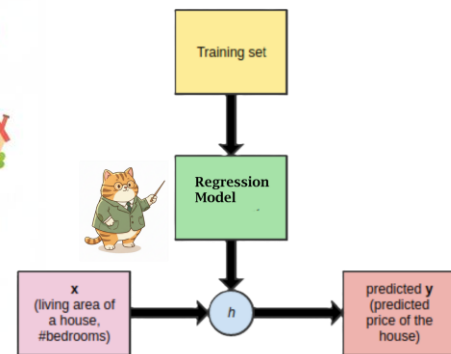
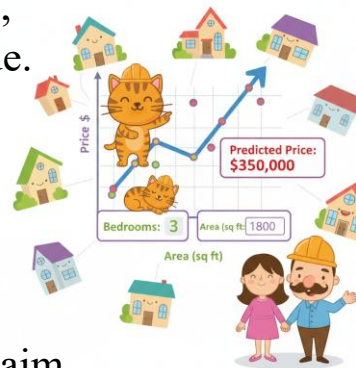
BScHons(Math), MSc(Industrial Math), MSc(Comp Sci), PhD



Defining the Regression Problem

- ❑ **The Predictive Task:** Regression is a Supervised Learning task focused on predicting a continuous numerical output (the target y) based on one or more input variables (the features $\mathbf{x}$).
- ❑ **Dataset Components:** Our training data consists of m examples, where each example $(\mathbf{x}^{(i)}, y^{(i)})$ is a complete, known pair of features and its corresponding target value.
- ❑ **Features ($\mathbf{x}$):** These are the measurable, independent variables used for prediction (e.g., area, number of bedrooms).
- ❑ **Target (y):** This is the dependent, continuous value we aim to predict (e.g., house price). The goal is to learn a mapping function $h: \mathbf{x} \rightarrow y$.

Living area (feet ²)	#bedrooms	Price (1000\$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
:	:	:



Simple Vs. Multiple Linear Regression

- ❑ **Simple Linear Regression (SLR):** Involves a single input feature (x_1) predicting the target y . This is primarily used for introductory concepts and basic trend analysis

$$\hat{y} = h(x) = w_0 + w_1x_1$$

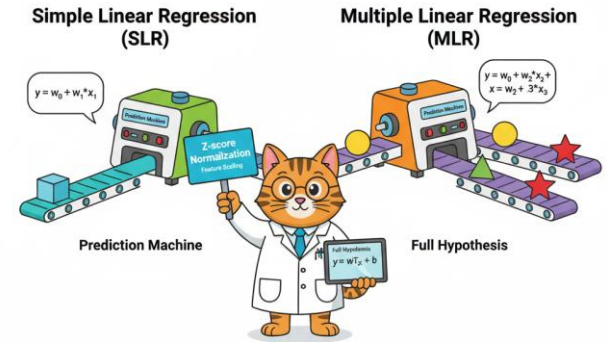
- ❑ **Multiple Linear Regression (MLR):** Utilizes two or more distinct features ($x_1, x_2, \dots, x_n$) to predict the target y . This is the standard, practical approach for real-world complexity.

$$\hat{y} = h(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

- ❑ **Best Practice:** Feature Scaling: When features have vastly different scales (e.g., area in ft^2 vs. number of rooms), we must standardize them (e.g., using Z-score normalization) to ensure faster and more stable training.
- ❑ **The Full Hypothesis:** The MLR hypothesis is a vector dot product:

$$\hat{y} = h(\mathbf{x}) = \sum_{i=0}^n w_i x_i = \mathbf{w}^T \mathbf{x}$$

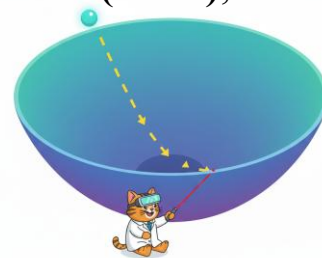
where $\mathbf{w}$ and $\mathbf{x}$ are vectors of weights and features, respectively. Also, $x_0 = 1$.



Learning the Parameters: Gradient Descent

- ❑ **The Problem:** We need an efficient way to find the parameter values (weights $\mathbf{w}$) that minimize our **Cost Function**, $J(\mathbf{w})$.
- ❑ **The Cost Function**, $J(\mathbf{w})$: For Linear Regression, this is the **Mean Squared Error (MSE)**, which measures the average squared error across m training examples:

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$



- ❑ **Gradient Descent (GD):** An iterative optimization algorithm that finds the minimum of $J(\mathbf{w})$ by repeatedly moving in the direction of the steepest descent.
- ❑ **The Update Rule:** In each step, we update every parameter w_j by subtracting the gradient scaled by the learning rate (α):

$$w_j^{new} = w_j^{old} - \alpha \frac{\partial}{\partial w_j} J(\mathbf{w})$$

Deriving the LMS Update Rule from Gradient

- ❑ **The Gradient (Partial Derivative):** The partial derivative of $J(\mathbf{w})$ w.r.t. a single weight w_j is calculated:

$$\frac{\partial}{\partial w_j} J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}) x_j^{(i)}$$

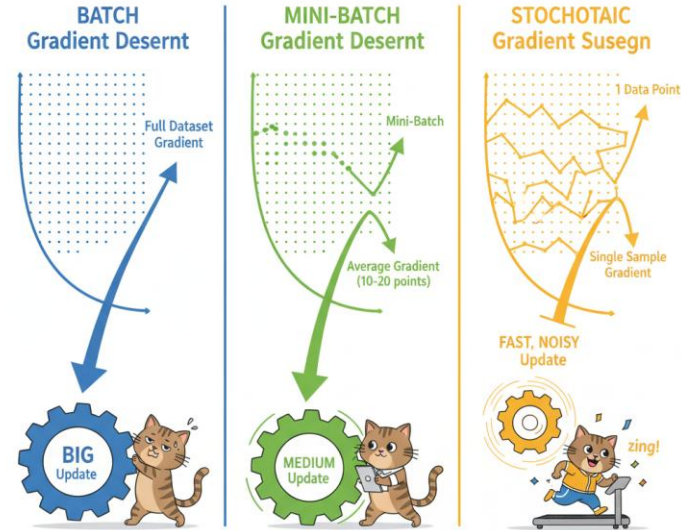
- ❑ **The LMS/Widrow-Hoff Update:** Substituting the derivative into the GD rule gives the **Mini-Batch/Batch** form:

$$w_j^{new} = w_j^{old} - \frac{\alpha}{m} \sum_{i=1}^m (\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}) x_j^{(i)}$$

- ❑ **The Stochastic (per-example) form is:**

$$w_j^{new} = w_j^{old} - \alpha (\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}) x_j^{(i)}$$

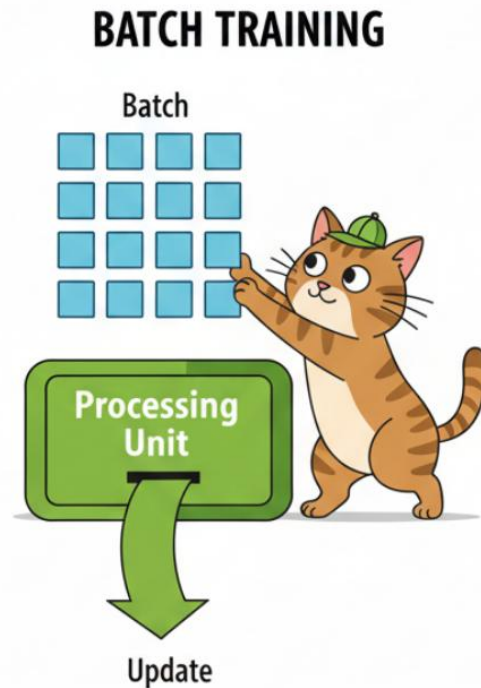
GRADIENT DESCENT ALGORITHMS



Loss Function = Curved Surface
Sphere = Model Parameters

Batch Training: The Stable Approach

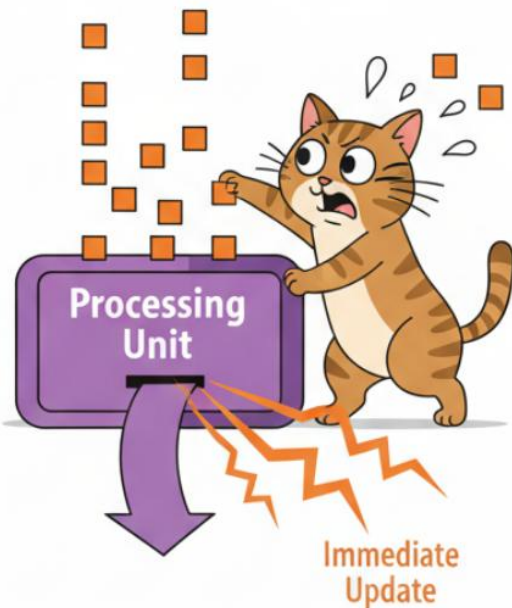
- ❑ **Definition:** The model updates its parameters using the averaged gradient calculated from a fixed batch (subset) of training examples simultaneously.
- ❑ **Process:** Divide data into batches → Compute errors/gradients for all samples in the batch → Average the gradients → Update parameters once per batch.
- ❑ **Advantage:** Parallelism & Stability: Leverages parallel computing (GPUs) for faster training and provides stable, low-variance updates that smooth out noise and outliers.
- ❑ **Drawback:** Memory & Convergence: Requires significant memory to hold the batch and intermediate results; can be slower to converge than smaller-step methods.



Incremental Training: The Adaptive Approach

- ❑ **Definition:** The model updates its parameters after processing **one single example** at a time, continuously adapting as new data arrives. (Also known as **Online or Stochastic Learning**).
- ❑ **Process:** Initial training on a small set → Present **one new data point** → Calculate prediction error → **Update parameters immediately** → Repeat for next new point.
- ❑ **Advantage: Real-Time & Adaptability:** Ideal for **streaming, evolving, or massive datasets** by not requiring the entire dataset in memory; quickly adapts to **concept drift** (changes in data properties over time).
- ❑ **Drawback: Sensitivity & Noise:** Highly sensitive to noise or outliers since each example directly influences the update; **convergence path is noisy** (high variance).

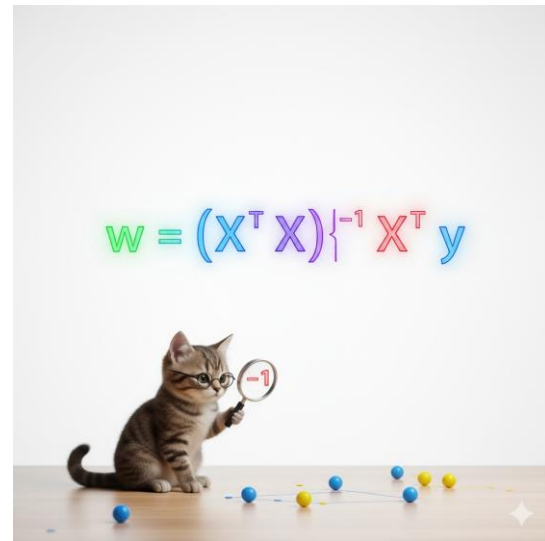
INCREMENTAL TRAINING



The Normal Equation: A Closed-Form Solution

- ❑ **The Alternative:** The Normal Equation is a **closed-form solution** that directly computes the optimal parameters $\mathbf{w}$ by minimizing the Cost Function $J(\mathbf{w})$ **analytically**, without needing iteration.
- ❑ **The Method:** We minimize the Mean Squared Error (MSE) loss by setting its **partial derivatives** with respect to every parameter w_j to zero. This is the condition for finding the global minimum.
- ❑ **Matrix Setup:** The training data is organized into a design matrix $\mathbf{X}$ ($m \times (n + 1)$) and the targets into a vector $\mathbf{y}$ ($m \times 1$).
 - ❖ $\mathbf{X}$ contains all m training examples as rows, with an added column of ones for the bias term.
 - ❖ $\mathbf{y}$ contains all m target values.
- ❑ **The Normal Equation Formula:** The solution for the optimal parameter vector $\mathbf{w}$ is directly given by the following matrix operation:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$



Gradient Descent vs. Normal Equation: The Scalability Trade-off

- ❑ **Computational Cost of Inverse:** The Normal Equation requires calculating the matrix inverse $(X^T X)^{-1}$, which is computationally expensive (typically $O(n^3)$), making it infeasible when the number of features (n) is large (e.g., $n > 10,000$).
- ❑ **Scalability and Data Size:** Gradient Descent (especially Mini-Batch GD) is computationally linear ($O(n \cdot m)$ or $O(n)$ per step) and excels with massive datasets (m). The Normal Equation struggles when n is large.
- ❑ **Memory Efficiency:** GD variants (like SGD) require significantly less memory because they only process a small batch or a single example at a time. The Normal Equation requires storing and manipulating the entire large design matrix X in memory.
- ❑ **Extensibility (The "Model Knob"):** Gradient Descent is highly extensible and modular. It easily accommodates complex additions like regularization terms (L1/L2) and is the only viable method for non-linear models and deep neural networks.

NORMAL EQUATION



GRADIENT DESCENT



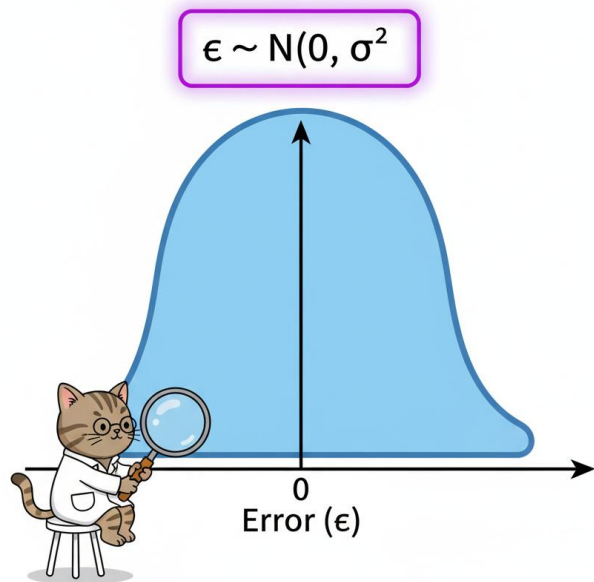
Probabilistic Foundation of Linear Regression

- ❑ **The Model:** We assume the target variable $y^{(i)}$ is related to the input features $\mathbf{x}^{(i)}$ via a linear relationship plus an error term $\epsilon^{(i)}$:

$$y^{(i)} = w^T \mathbf{x}^{(i)} + \epsilon^{(i)}$$

- ❑ **The Error Term ($\epsilon^{(i)}$):** This captures the influence of:
 1. **Unmodeled effects:** Features left out of the model.
 2. **Random Noise:** Inherent unpredictability.
- ❑ **The Key Assumption:** We assume these errors $\epsilon^{(i)}$ are **Independently and Identically Distributed (IID)** according to a **Normal (Gaussian) Distribution** with a mean of zero and variance σ^2 .

$$\epsilon^{(i)} \sim N(0, \sigma^2)$$



The Likelihood Function

- ❑ **Conditional Probability:** Based on the normal error assumption, the probability distribution of $y^{(i)}$ given $\mathbf{x}^{(i)}$ (parameterized by $\mathbf{w}$) is:

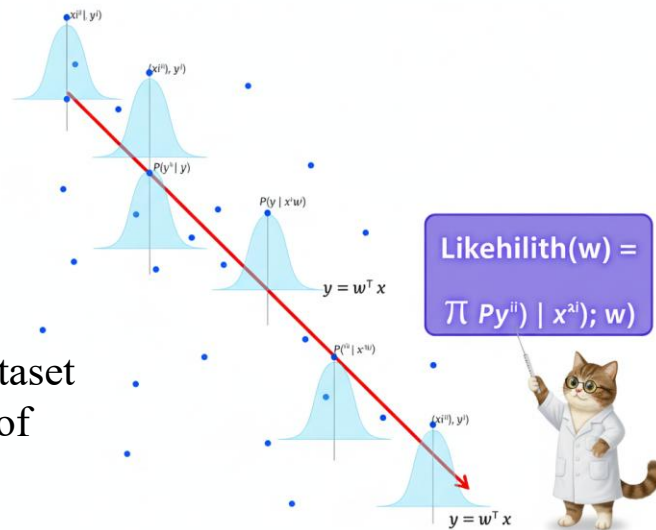
$$P(y^{(i)}|\mathbf{x}^{(i)}; \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2}{2\sigma^2}}$$

- ❑ **Definition: Likelihood Function ($L(\mathbf{w})$):** Given the entire dataset ($\mathbf{X}$ and $\mathbf{y}$), the probability of observing this data as a function of the unknown weights $\mathbf{w}$ is called the Likelihood Function.

$$L(\mathbf{w}) = P(\mathbf{y}|\mathbf{X}; \mathbf{w})$$

- ❑ **IID Factorization:** Due to the **independence assumption** of the errors ($\epsilon^{(i)}$), the total likelihood is the **product** of the probabilities of all individual training examples:

$$L(\mathbf{w}) = \prod_{i=1}^m P(y^{(i)}|\mathbf{x}^{(i)}; \mathbf{w})$$

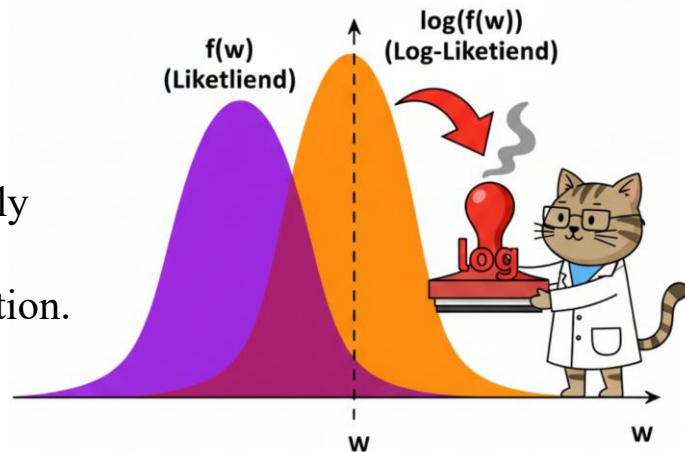


Maximum Likelihood and Log-Likelihood

- ❑ **Maximum Likelihood Principle:** We must choose the weight vector $\mathbf{w}$ that maximizes the likelihood function $L(\mathbf{w})$.

$$\mathbf{w}_{MLE} = \arg \max_{\mathbf{w}} L(\mathbf{w})$$

- ❑ **Simplification: Log-Likelihood ($\ell(\mathbf{w})$):** It is mathematically simpler to maximize the logarithm of the likelihood, $\ell(\mathbf{w}) = \log L(\mathbf{w})$, because the logarithm is a strictly increasing function.
 - ❖ Maximizing $L(\mathbf{w})$ is equivalent to maximizing $\ell(\mathbf{w})$.
 - ❖ The logarithm converts the complex product ($\prod$) into a simple sum ($\sum$).



- ❑ **The Log-Likelihood Equation (Expanded):**

$$\ell(\mathbf{w}) = m \log \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2\sigma^2} \sum_{i=1}^m (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

The Link to Least Squares (MSE)

$$\ell(\mathbf{w}) = m \log \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2\sigma^2} \sum_{i=1}^m (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

- ❑ **Maximization vs. Minimization:** The first term in the equation is a **constant** with respect to $\mathbf{w}$. The second term is multiplied by a **negative coefficient** ($-1/2\sigma^2$).
- ❑ **The Equivalence:** Maximizing the log-likelihood $\ell(\mathbf{w})$ is equivalent to **minimizing the remaining positive term**.

$$\arg \max_{\mathbf{w}} \ell(\mathbf{w}) \Leftrightarrow \arg \min_{\mathbf{w}} \left[\sum_{i=1}^m (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2 \right]$$

- ❑ **Conclusion:** The term we are minimizing is exactly the **Least Squares Cost Function** (our original $J(\mathbf{w})$), scaled by $\frac{1}{2}$ and without the average $\frac{1}{m}$.



Preventing Overfitting: Introduction to Regularization

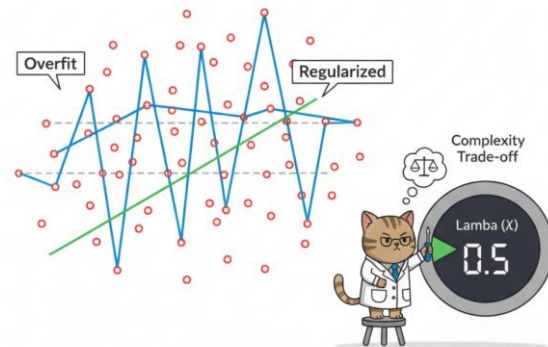
❑ **The Problem (Overfitting):** A model performs brilliantly on training data but fails on unseen test data. This often occurs when weights (w) become excessively large to perfectly fit noise in the training set.

❑ **The Solution (Regularization):** A technique that modifies the Cost Function ($J(\mathbf{w})$) by adding a penalty term based on the magnitude (size) of the weights.

❑ **Regularized Cost Function (J_{reg}):** We seek to minimize the original loss plus the **penalty**:

$$\min_{\mathbf{w}} J_{reg}(\mathbf{w}) = J(\mathbf{w}) + \lambda \cdot \text{Penalty}(\mathbf{w})$$

❑ **The Hyperparameter (λ):** Lambda (λ) is the regularization strength. A large λ forces weights to be smaller, increasing bias but improving generalization (decreasing variance). A $\lambda = 0$ reverts to the original unregularized model.



Regularization Penalty Terms: L1 vs. L2

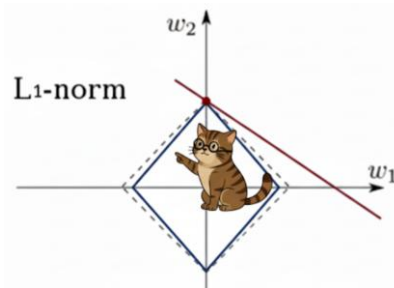
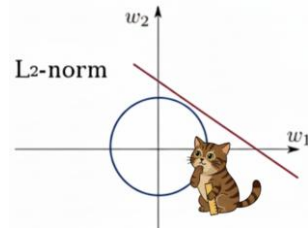
- ❑ **L2 Regularization (Ridge Regression):** The penalty term is the sum of the **squared** magnitudes of all weights. This forces all weights to shrink toward zero, but none are entirely eliminated.

$$J_{Ridge}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_{j=0}^n w_j^2$$

- ❑ **L1 Regularization (Lasso Regression):** The penalty term is the sum of the **absolute** magnitudes of all weights. This has the unique effect of forcing some weights to become **exactly zero**.

$$J_{Lasso}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_{j=0}^n |w_j|$$

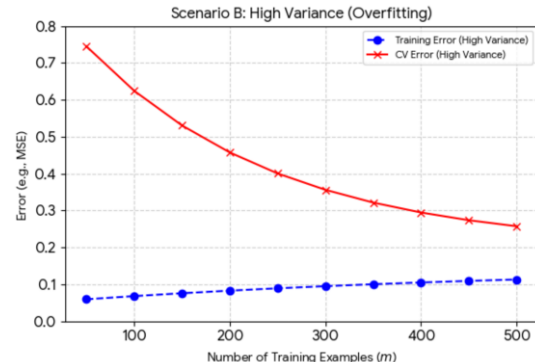
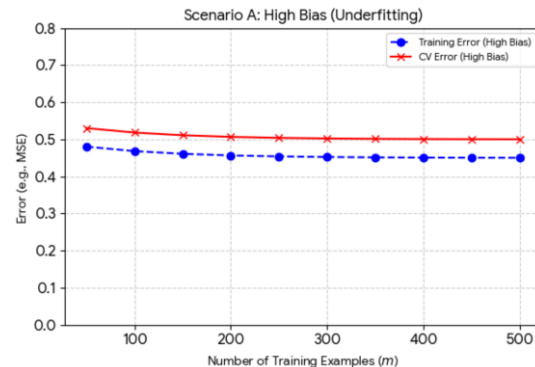
- ❑ **Elastic Net:** A hybrid approach that combines both L1 and L2 penalties. This often yields the best practical performance by achieving both weight shrinkage (L2) and feature selection (L1).



- ❑ **Effect on Model:** Regularization effectively restricts the model's complexity by keeping the solution simple and improving **generalization** on unseen data.

Detecting Overfitting and Underfitting

- ❑ **The Goal:** Diagnose the Bias-Variance Trade-off by comparing model performance on the Training Set (known data) and the Cross-Validation (CV) Set (unseen data).
- ❑ **High Bias (Underfitting):** The model is too simple (e.g., using a straight line for curved data).
 - ❖ **Diagnostic:** High error on both the Training Set and the CV Set. The model is fundamentally poor.
- ❑ **High Variance (Overfitting):** The model is too complex and memorizes the noise in the training data.
 - ❖ **Diagnostic:** Low error on the Training Set but significantly higher error on the CV Set. The model fails to generalize.
- ❑ **Diagnostic Tool (Learning Curves):** Plots the Training Error and CV Error as a function of the **number of training examples (m)**. This visually identifies whether adding more data will improve the model.



Model Suitability Check: Metrics and Assumptions

❑ **Accuracy Metrics (Quantitative):** Use R^2 to check the variance explained (goodness of fit) and **RMSE/MAE** to quantify the average prediction error magnitude.

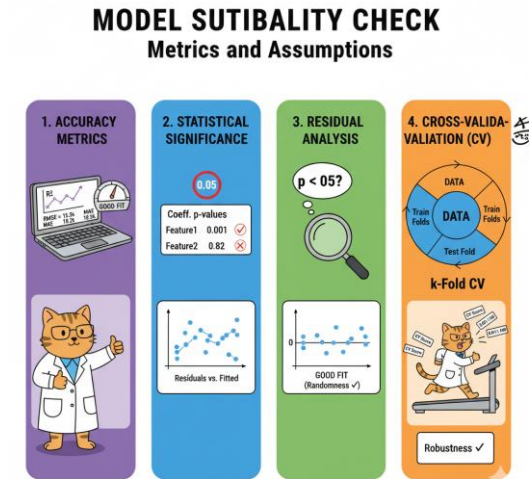
❑ **Statistical Significance:** Check **p-values** for each feature's coefficient. A low p-value (<0.05) confirms the feature has a statistically significant relationship with the target.

❑ **Assumption Check:**

Residual Analysis: Plot the residuals (prediction errors) to verify the core statistical assumptions of the model.

❑ **Best Practice:**

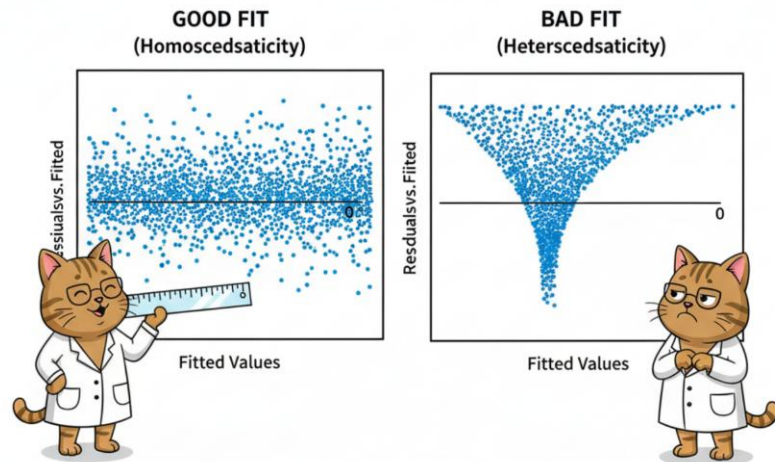
Cross-Validation (CV): Use **k-Fold Cross-Validation** to repeatedly test the model on different, unseen data partitions. This provides the most robust measure of its generalization ability.



Visual Diagnosis: Interpreting Residual Plots

- ❑ **Plot 1 (Residuals vs. Fitted Values):** Plot the error ($\hat{y} - y$) against the model's prediction ($\hat{y}$).

- ❖ **Goal:** The points must be randomly scattered around the zero-horizontal line.
- ❖ **Bad Pattern:** A U-shape, funnel shape, or any visible pattern suggests the model is biased—either a **non-linear relationship** exists, or the error variance is **non-constant** (heteroscedasticity).



- ❑ **Plot 2 (Normal Q-Q Plot of Residuals):** Plots the observed residuals against a theoretical normal distribution.
- ❖ **Goal:** The points should fall perfectly along a straight diagonal line.
- ❖ **Bad Pattern:** Points deviating significantly from the line indicate the **errors are not normally distributed**, invalidating many standard statistical confidence tests.

Fixing Model Bias: Action Plan for Bad Residuals

❑ Problem 1: Non-Linearity (Systematic Pattern like a U-shape)

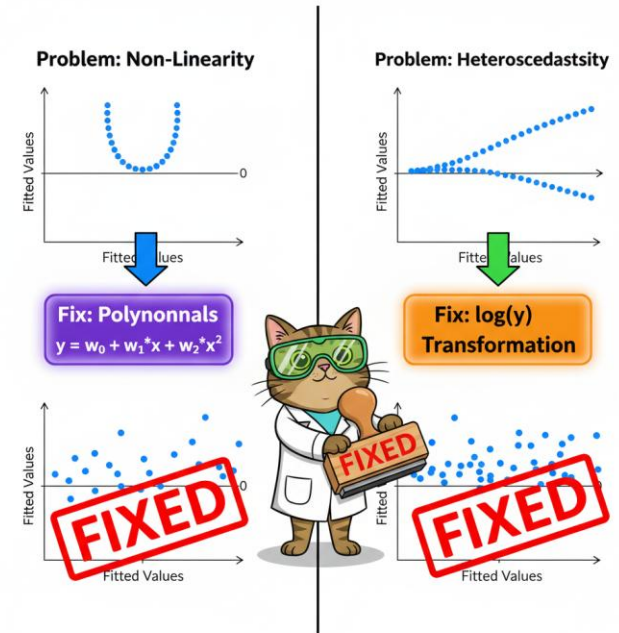
- ❖ **Action: Introduce polynomial features.** Add higher-order terms of existing features (e.g., x^2 , x^3) to the model.
- ❖ **Action: Consider switching to a fundamentally non-linear model** (e.g., Decision Trees, Neural Networks) that can inherently capture curves.

❑ Problem 2: Non-Constant Variance (Heteroscedasticity like a Funnel-Shape)

- ❖ **Action: Transform the target variable (y)** using a non-linear function, such as the logarithm ($\log(y)$) or square root. This often stabilizes the variance.
- ❖ **Action: Use Weighted Least Squares (WLS)**, which explicitly gives less weight to the data points associated with the larger errors (the wide end of the funnel).

❑ Problem 3: Non-Normal Errors (Deviation on Q-Q Plot)

- ❖ **Action: Remove significant outliers** that may be skewing the error distribution.
- ❖ **Action: Again, transform the target variable (y)** or highly skewed predictor variables (x).



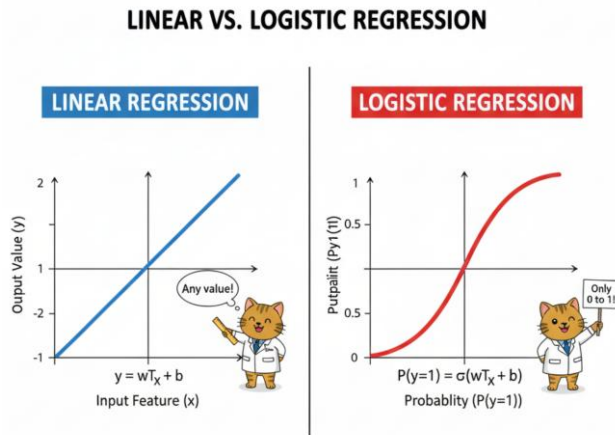
Classification: The Need for Logistic Regression

- ❑ **Problem Shift:** We move from predicting a **continuous value** (e.g., house price) to predicting a **discrete variable** (a class or category).

❖ **Examples:** Is an image '0' or '1'? Is an email **Spam** or **Legitimate**?

- ❑ **The Limitation of Linear Regression:** Standard Linear Regression outputs values from $-\infty$ to $+\infty$, which are meaningless as probabilities in a classification task.

- ❑ **The Solution:** We need a hypothesis function $h_w(x)$ that is **bounded** between 0 and 1 to represent the probability of an event occurring.
- ❑ **Logistic Regression:** This simple classification algorithm achieves this by using the **Sigmoid (or Logistic) function**.



The Logistic Hypothesis: The Sigmoid Function

- ❑ **Hypothesis Function:** The Logistic Regression hypothesis is the linear model wrapped inside the Sigmoid function $g(z)$:

$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x})$$

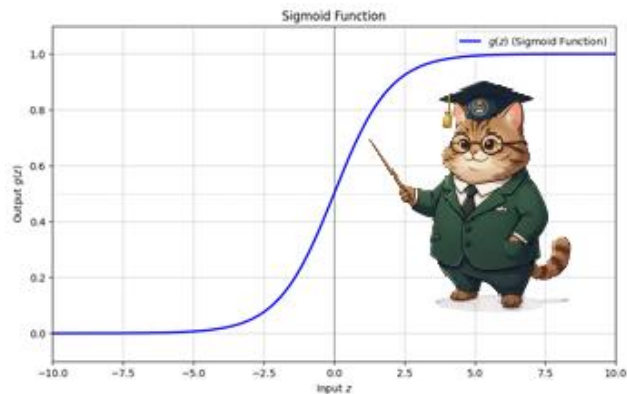
where $z = \mathbf{w}^T \mathbf{x} = w_0 + \sum_{j=1}^n w_j x_j$

- ❑ **The Sigmoid Function:** Also known as the **Logistic Function**:

$$g(z) = \frac{1}{1 + e^{-z}}$$

- ❑ **Properties:**

- ❖ As $z \rightarrow \infty, g(z) \rightarrow 1$.
- ❖ As $z \rightarrow -\infty, g(z) \rightarrow 0$.
- ❖ Crucially, the output $h_{\mathbf{w}}(\mathbf{x})$ is always strictly bounded between 0 and 1.

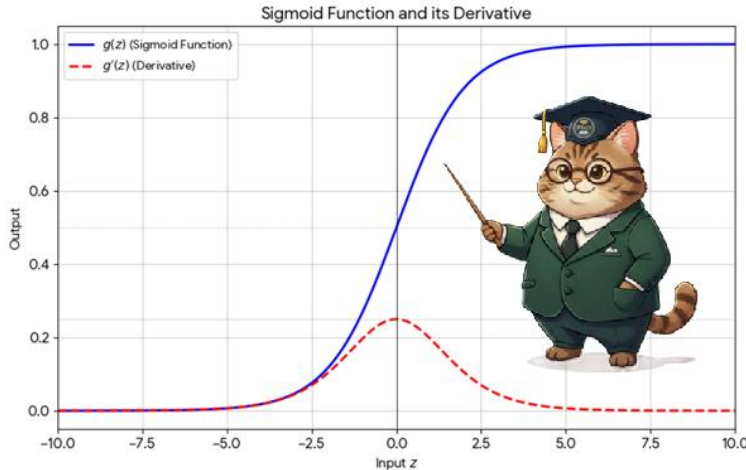


Derivative of the Logistic Function (For Gradient Descent)

- ❑ **Necessity:** To train the model using Gradient Descent, we need to calculate the partial derivatives of the cost function, which requires the derivative of the Sigmoid function itself.
- ❑ **The Derivative:** The derivative of the Sigmoid function, $g'(z)$, simplifies remarkably:

$$g'(z) = \frac{d}{dz} \left[\frac{1}{1 + e^{-z}} \right] = g(z)[1 - g(z)]$$

- ❑ **Key Property:** This compact form is one of the key algebraic properties that makes Logistic Regression (and later, Neural Networks that use the Sigmoid activation) easy to optimize via Gradient Descent.



$$\begin{aligned} g'(z) &= \frac{d}{dz} \left[\frac{1}{1 + e^{-z}} \right] \\ &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\ &= \frac{1}{(1 + e^{-z})} \left[1 - \frac{1}{(1 + e^{-z})} \right] \\ &= g(z) [1 - g(z)] \end{aligned}$$

Fitting Parameters: Maximum Likelihood

- ❑ **Probabilistic Model:** We assume that the probability of the target being 1 or 0 is given directly by our hypothesis:

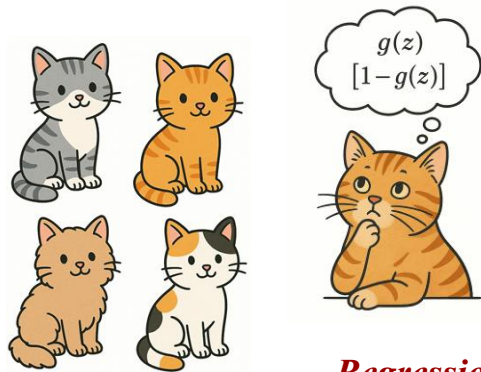
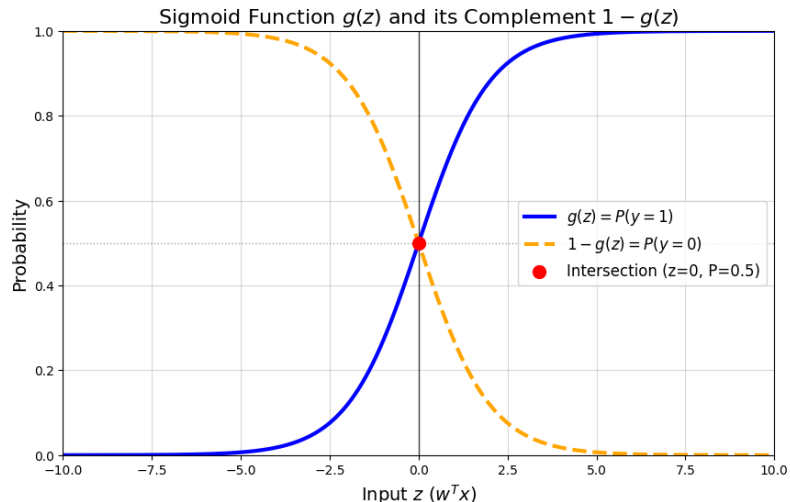
$$P(y = 1|\mathbf{x}; \mathbf{w}) = h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x})$$

$$P(y = 0|\mathbf{x}; \mathbf{w}) = 1 - h_{\mathbf{w}}(\mathbf{x}) = 1 - g(\mathbf{w}^T \mathbf{x})$$

- ❑ **Compact Likelihood:** These two statements can be combined into a single compact expression for the probability mass function of y :

$$P(y|\mathbf{x}; \mathbf{w}) = h_{\mathbf{w}}(\mathbf{x})^y [1 - h_{\mathbf{w}}(\mathbf{x})]^{(1-y)}$$

- ❖ If $y = 1$, the first term remains, and the second term becomes 1.
- ❖ If $y = 0$, the first term becomes 1, and the second term remains.



The Full Likelihood Function (Classification)

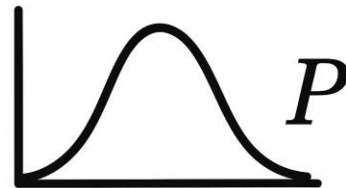
- ❑ **Likelihood Function:** Assuming all m training examples were generated **independently**, the total likelihood $L(\mathbf{w})$ of the entire dataset is the product of the individual probabilities:

$$L(\mathbf{w}) = P(\mathbf{y}|\mathbf{X}; \mathbf{w}) = \prod_{i=1}^m P(y^{(i)}|\mathbf{x}^{(i)}; \mathbf{w})$$

- ❑ **Expanded Likelihood:** Substituting the compact form from the previous slide:

$$L(\mathbf{w}) = \prod_{i=1}^m [h_{\mathbf{w}}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})]^{(1-y^{(i)})}$$

- ❑ **Objective:** We seek to find the weight vector $\mathbf{w}$ that maximize this likelihood function $L(\mathbf{w})$.



The Log-Likelihood and Cross-Entropy Loss

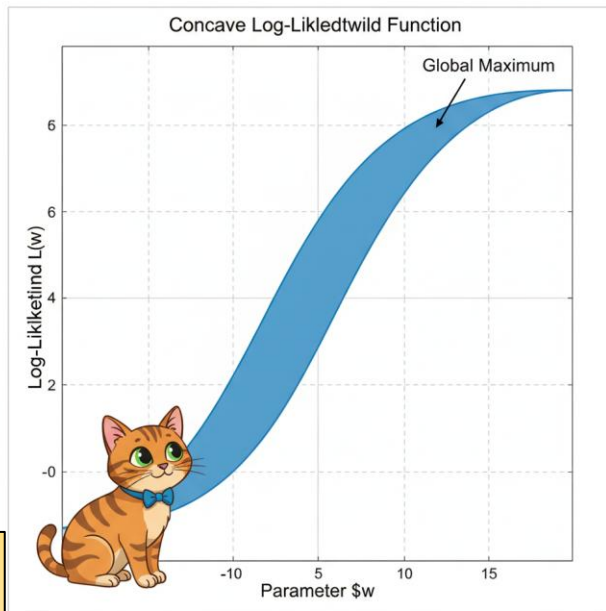
- ❑ **Log-Likelihood ($\ell(\mathbf{w})$):** Maximizing the log-likelihood is equivalent to maximizing the likelihood:

$$\ell(\mathbf{w}) = \log L(\mathbf{w})$$

- ❑ **The Cross-Entropy Sum:** Applying the logarithm converts the product into a sum, resulting in the **Log-Likelihood** (which is the negative of the **Cross-Entropy Loss**):

$$\ell(\mathbf{w}) = \sum_{i=1}^m [y^{(i)} \log h_{\mathbf{w}}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\mathbf{w}}(x^{(i)}))]$$

- ❑ **Convexity:** This Log-Likelihood function is **concave** (meaning the negative of it, the Cross-Entropy Loss, is **convex**), guaranteeing that any local maximum found by Gradient Ascent is also the global maximum.

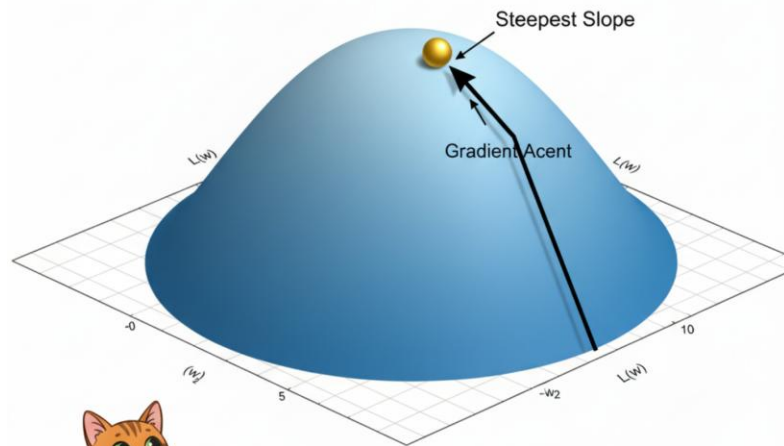


Optimization: Gradient Ascent

- ❑ **The Objective:** Since we are maximizing the Log-Likelihood function $\ell(\mathbf{w})$, we use **Gradient Ascent**.
- ❑ **Gradient Ascent Rule:** Instead of subtracting the gradient (descent), we **add** the gradient (ascent) to climb to the maximum:

$$w_j^{new} = w_j^{old} + \alpha \frac{\partial}{\partial w_j} \ell(\mathbf{w}) \text{ for } j = 0, 1, \dots, n$$

- ❑ **Differentiating the Log-Likelihood:** To implement the rule, we must first calculate the partial derivative of $\ell(\mathbf{w})$ with respect to each weight w_j .



The Gradient of the Log-Likelihood

- ❑ **Applying the Chain Rule:** The partial derivative of the Log-Likelihood with respect to w_j is calculated using the chain rule on the sum.

$$\frac{\partial}{\partial w_j} \ell(\mathbf{w}) = \sum_{i=1}^m \left[\frac{\partial}{\partial w_j} y^{(i)} \log h_w(x^{(i)}) + \frac{\partial}{\partial w_j} (1 - y^{(i)}) \log(1 - h_w(x^{(i)})) \right]$$

- ❑ **Key Simplification (Algebraic Magic):** After applying the chain rule, utilizing the derivative property $g'(z) = g(z)[1 - g(z)]$, and simplifying, the terms collapse beautifully.
- ❑ **Final Simplified Gradient (The Punchline):** After all the algebraic simplification (from the previous slides), the derivative of the Log-Likelihood is:

$$\frac{\partial}{\partial w_j} \ell(\mathbf{w}) = \sum_{i=1}^n [y^{(i)} - h_w(x^{(i)})] x_j^{(i)}$$



Update Rule

- ❑ **Gradient Ascent Update Rule:** Substituting this back into the Gradient Ascent formula:

$$w_j^{new} = w_j^{old} + \alpha \sum_{i=1}^n [y^{(i)} - h_w(x^{(i)})] x_j^{(i)} \text{ for } j = 0, 1, \dots, n$$

- ❑ **Similarity to LMS:** This final update rule is **identical in form** to the LMS (Least Mean Squares) update rule for Linear Regression.
- ❑ **The Crucial Difference:** The difference is that $h_w(\mathbf{x}^{(i)})$ is now the **non-linear Sigmoid function** of $\mathbf{w}^T \mathbf{x}^{(i)}$, making the overall Logistic Regression algorithm non-linear.

LMS Update Rule



Logistic Regression Update Rule



Note: Model is linear on the left, Sigmoid on the right.

Classification: Making a Decision

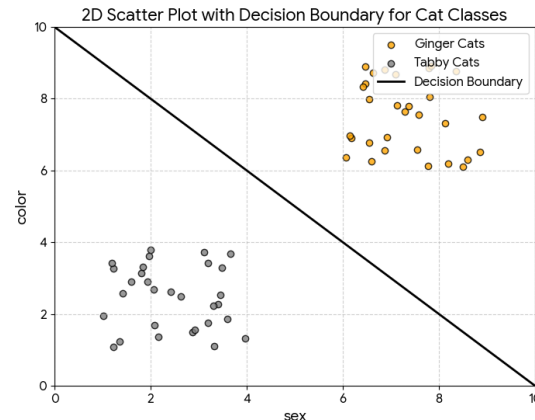
- ❑ **Hypothesis Output:** The model's output $h_w(\mathbf{x})$ is an estimated probability: $P(y = 1 \mid \mathbf{x}; \mathbf{w})$.
- ❑ **The Decision Rule:** To classify an input into a discrete class (0 or 1), we set a **threshold**, typically 0.5:

- ❖ Predict $y = 1$ if $h_w(\mathbf{x}) \geq 0.5$
- ❖ Predict $y = 0$ if $h_w(\mathbf{x}) < 0.5$

- ❑ **The Decision Boundary:** The boundary is the line (or hyperplane) where the probability is exactly 0.5. Since $g(z)=0.5$ when $z=0$, the decision boundary is defined by the equation:

$$\mathbf{w}^T \mathbf{x} = 0$$

- ❑ **Nature of the Boundary:** Logistic Regression is inherently a **linear classifier**. The decision boundary is always a straight line in the feature space, unless polynomial features are explicitly added.



Evaluation: The Confusion Matrix

- ❑ **Need for Classification Metrics:** Accuracy alone is insufficient for classification. We need to evaluate the types of errors a model makes.
- ❑ **The Confusion Matrix:** A table used to describe the performance of a classification model on a set of test data for which the true values are known.
- ❑ **Key Derived Metrics:**
 - ❖ **Accuracy:** $(TP+TN)/\text{Total}$ (Overall correct predictions)
 - ❖ **Precision:** $TP/(TP+FP)$ (When model predicts '1', how often is it correct?)
 - ❖ **Recall (Sensitivity):** $TP/(TP+FN)$ (Out of all actual '1's, how many did the model find?)

	Positive (Predicted)	Negative (Predicted)
Positive (Actual)	TRUE POSITIVE (TP) Correctly predicted positive	FALSE NEGATIVE (FN) Incorrectly predicted negative
Negative (Actual)	FALSE POSITIVE (FP) Incorrectly predicted positive	TRUE NEGATIVE (TN) Correctly predicted negative

 Confusion Matrix

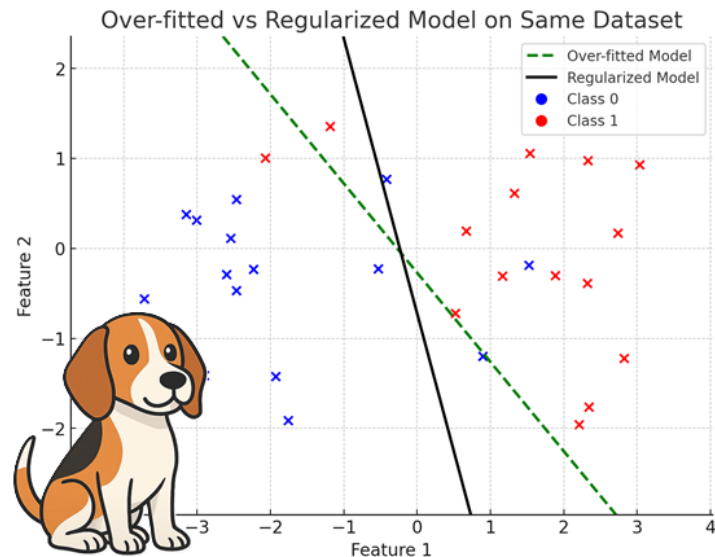
A classifier that is 99% accurate is only 0% useful if it is wrong on the 1% that matters.

Regularization for Logistic Regression

- ❑ **The Problem:** Logistic Regression, like Linear Regression, can suffer from **overfitting** when weights ($\mathbf{w}$) become excessively large, making the model overly complex.
- ❑ **The Solution:** We add a penalty term to the Log-Likelihood (or subtract it from the negative Log-Likelihood/Cross-Entropy Loss) to shrink the weights.
- ❑ **Regularized Objective:** We aim to maximize the penalized Log-Likelihood:

$$\arg \max_{\mathbf{w}} [\ell(\mathbf{w}) - \lambda \cdot \text{Penalty}(\mathbf{w})]$$

L2 Regularization (Ridge): Penalty is the sum of squared weights. This is common and stable.



L1 Regularization (Lasso): Penalty is the sum of absolute weights. This performs **feature selection** by driving some weights exactly to zero.

Softmax Regression: The Multi-Class Problem

- ❑ **The Limitation:** Logistic Regression handles only two classes ($y \in \{0,1\}$).
- ❑ **The Need:** We need a model when the response variable y can take on any one of K values, where $K > 2$.
i.e., $y \in \{1,2, \dots, K\}$
- ❑ **Examples:**
 - ❖ Classifying an email as **Spam**, **Urgent**, or **Normal** ($K=3$).
 - ❖ Classifying dog images into ten different breeds (e.g., Labrador, Poodle, German Shepherd, etc.) ($K=10$).
- ❑ **Solution: Softmax Regression** (also known as Multinomial Logistic Regression) generalizes the principles of Logistic Regression to K classes.



The Softmax Function (Definition)

- ❑ **The Softmax Goal:** The Softmax function (σ) takes a K-dimensional vector of arbitrary real values and "squashes" it into a K-dimensional vector of real values in the range $[0,1]$ that **sum up to 1**.

$$\text{softmax}: \mathbb{R}^K \rightarrow [0,1]^K$$

- ❑ **The Softmax Equation:** For the j-th element of the output vector $\mathbf{z}$:

$$z_j = \text{softmax}(\mathbf{x})_j = \frac{e^{x_j}}{\sum_{k=1}^K e^{x_k}}, \text{ for } j = 1, 2, \dots, K$$

- ❑ **Mechanism:**

1. The **exponential function** (e^x) makes all scores positive and gives greater weight to larger scores.
2. The **denominator** acts as a normalization term, ensuring the K output probabilities sum exactly to 1.

Example:

Let's take $\mathbf{x} = [1, 2, 3, 4, 1, 2, 3]^T$, then

$$\begin{aligned} z_1 &= \frac{e^{x_1}}{e^{x_1} + e^{x_2} + \dots + e^{x_7}} \\ &= \frac{e^1}{e^1 + e^2 + \dots + e^3} \\ &= 0.024 \end{aligned}$$

Similarly, we can find other z values.

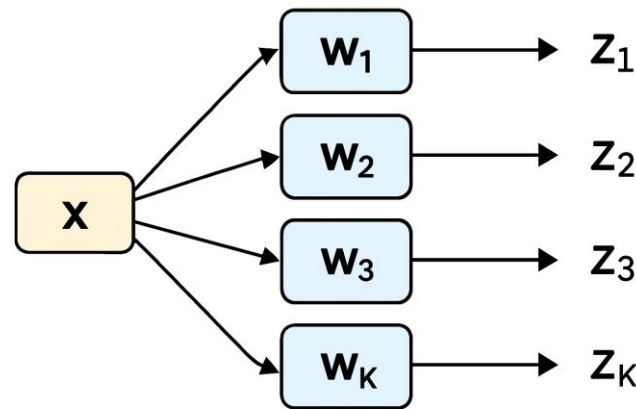
And finally we get $\mathbf{z}$ as $[0.024, 0.064, 0.175, 0.475, 0.024, 0.064, 0.175]$.

Softmax Hypothesis: Model Parameters

- ❑ **Hypothesis Goal:** Estimate the probability for each class k : $P(y = k \mid \mathbf{x})$, for $k = 1, 2, \dots, K$.
- ❑ **The Model:** Softmax runs K parallel linear models, one for each class.
 - ❖ Each class k has its own weight vector: $\mathbf{w}_k \in \mathbb{R}^{n+1}$.
 - ❖ The entire model is parameterized by the weight matrix $\mathbf{W}$, which contains all K weight vectors.

$$\mathbf{W} = \begin{bmatrix} -\mathbf{w}_1^T & - \\ -\mathbf{w}_2^T & - \\ \vdots & \\ -\mathbf{w}_K^T & - \end{bmatrix} \quad (\text{a } K \times (n + 1) \text{ matrix})$$

- ❑ **Raw Scores:** The input to the Softmax function is the vector of linear scores: $\mathbf{W}^T \mathbf{x}$.



Softmax: The art of turning **relative advantage** into a **probability distribution**—amplifying the best choice while still acknowledging the rest.



Softmax Hypothesis: The Probability Vector

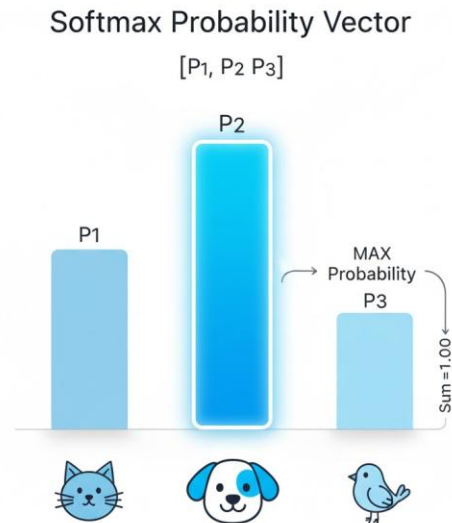
- ❑ **Hypothesis Form:** The final hypothesis is the Softmax function applied to the vector of linear scores:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{softmax}(\mathbf{W}^T \mathbf{x}) (= \mathbf{z})$$

- ❑ **The Probability Vector:** The hypothesis outputs a vector where each element is the estimated probability of the corresponding class:

$$h_{\mathbf{w}}(x) = \begin{bmatrix} P(y = 1 | \mathbf{x}; \mathbf{W}) \\ P(y = 2 | \mathbf{x}; \mathbf{W}) \\ \vdots \\ P(y = K | \mathbf{x}; \mathbf{W}) \end{bmatrix} = \frac{1}{\sum_{k=1}^K e^{\mathbf{w}_k^T \mathbf{x}}} \begin{bmatrix} e^{\mathbf{w}_1^T \mathbf{x}} \\ e^{\mathbf{w}_2^T \mathbf{x}} \\ \vdots \\ e^{\mathbf{w}_K^T \mathbf{x}} \end{bmatrix}$$

- ❑ **Final Prediction:** The model's prediction is the class k with the highest probability
 - ❖ i.e., $\max_k P(y = k | \mathbf{x}; \mathbf{W})$



Target Encoding: One-Hot Encoding (OHE)

❑ **The Problem with Integer Labels:** In a multi-class problem, assigning integer labels (e.g., Cat=1, Dog=2, Lion=3) suggests an artificial ordering or magnitude (e.g., Lion > Dog), which is incorrect for nominal categories.

❑ **One-Hot Encoding (OHE):** A process to convert categorical labels into a format suitable for machine learning, specifically for use with the Softmax Cost Function.

Class	One-Hot Vector
 Cat	[1, 0, 0]
 Dog	[0, 1, 0]
 Lion	[0, 0, 1]

❑ **Mechanism:** Each categorical label is converted into a vector of size K (number of classes), where:

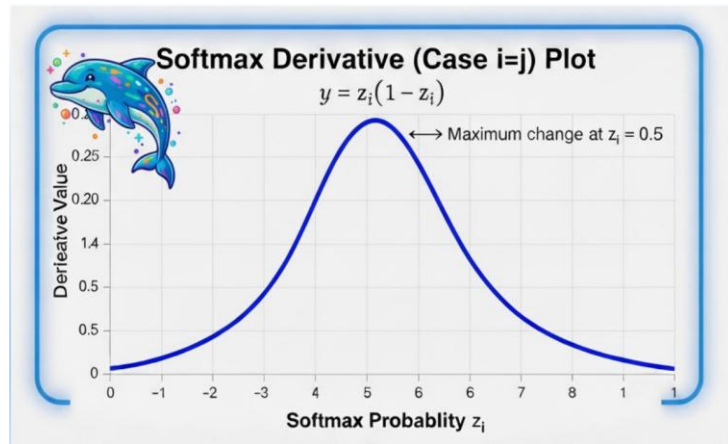
- ❖ The position corresponding to the true class is set to **1**.
- ❖ All other positions are set to **0**.

Derivative of Softmax (Case $i = j$)

- ❑ **Need for Derivative:** To train Softmax Regression, we use Gradient Ascent/Descent, which requires calculating the partial derivative of the Softmax output with respect to its input raw scores.
- ❑ **Case 1: Self-Influence ($i = j$):** This derivative shows how the output probability of a class i changes when its own raw score x_i is slightly increased.

$$\frac{\partial z_i}{\partial x_i} = z_i(1 - z_i)$$

- ❑ **Interpretation:** The derivative is identical in form to the Sigmoid function's derivative. This result is used in the overall chain rule calculation for the gradient. The change in the probability z_i is largest when z_i is near 0.5 and smallest when it's near the extremes (0 or 1).



Derivative of Softmax (Case $i \neq j$)

- ❑ **Case 2: Cross-Influence ($i \neq j$):** This derivative shows how the probability of a different class j changes when the raw score of class i is increased.

$$\frac{\partial z_j}{\partial x_i} = -z_j z_i$$

- ❑ **Interpretation:** The negative sign is key. Increasing the raw score for class i **decreases** the probability of every other class j .
- ❑ **Competitive Nature:** This term is what enforces the "competition" in Softmax. Since the probabilities must sum to 1, any gain in probability for one class must come at the expense of all other classes.

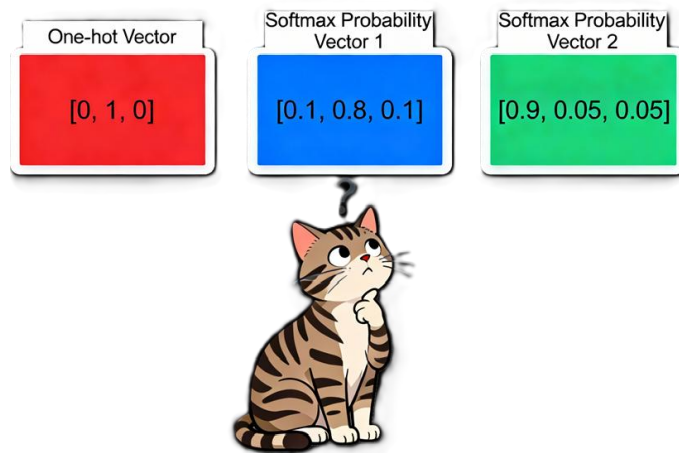


Four candidates stand on glowing platforms in a job selection scene. The central candidate shines brightest, showing the highest Softmax probability, while others fade slightly—illustrating how one's gain reduces the others' chances in a competitive, zero-sum setting.

The Need for Cross-Entropy Error

- ❑ **Hypothesis and Target are Vectors:** In Softmax Regression, both the model's output (probability vector $\mathbf{p}$) and the true label ($\mathbf{y}$) are **K-dimensional vectors**.

- ❖ **Hypothesis $\mathbf{p}$:** Contains floating-point probability values (e.g., $[0.1, 0.8, 0.1]$).
- ❖ **Target $\mathbf{y}$:** Is a **One-Hot Encoded (OHE)** vector (e.g., $[0, 1, 0]$), with a '1' in the position of the true class and '0's elsewhere.



- ❑ **The Problem with MSE:** Using the Sum of Squared Errors (MSE) between $\mathbf{p}$ and $\mathbf{y}$ is inefficient and non-optimal for probability distributions.
- ❖ MSE incorrectly penalizes the model for probabilities in the **zero positions** of the OHE vector.
- ❖ MSE creates a **non-convex** cost landscape when combined with the Softmax function, making optimization unreliable.

The Cross-Entropy Error Definition

- ❑ **Cross-Entropy Error $H(y,p)$:** This function measures the dissimilarity between the predicted probability distribution p and the true distribution y for a single training example.

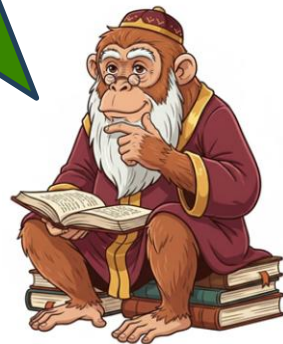
$$H(y, p) = - \sum_{j=1}^K y_j \log(p_j)$$

Interpretation: The error is solely determined by the **predicted probability of the true class**. If this probability is close to 1, the error is near zero. If it's near 0, the error approaches infinity.

- ❖ p_j : The model's predicted probability for class j (from Softmax).
- ❖ y_j : The true class indicator for class j (from the OHE vector).

- ❑ **The Simplification:** Because y is one-hot, the sum collapses to a single term.
 - ❖ Only the $\log(p_j)$ term corresponding to the true class (where $y_j = 1$) remains. All other terms (where $y_j = 0$) drop out.

$$H(y, p) = - \log(p_{\text{true class}})$$



The Softmax Cost Function (Loss)

- ❑ **Loss Function Definition:** The overall Cost Function $L(\mathbf{W})$ for Softmax Regression is the **average Cross-Entropy Error** over all m training examples.

$$L(\mathbf{W}) = \frac{1}{m} \sum_{i=1}^m H(\mathbf{y}^{(i)}, \mathbf{p}^{(i)}) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_j^{(i)} \log(p_j^{(i)})$$

- ❑ **The Full Cost Function:** Substituting the Softmax hypothesis $p_j = P(y = j | \mathbf{x}; \mathbf{W})$ into the equation:

$$L(\mathbf{W}) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_j^{(i)} \log \left[\exp(\mathbf{w}_j^T \mathbf{x}^{(i)}) / \sum_{k=1}^K \exp(\mathbf{w}_k^T \mathbf{x}^{(i)}) \right]$$

Optimization: This Cost Function $L(\mathbf{W})$ is convex, meaning it has a single global minimum. We cannot solve for the minimum analytically (like the Normal Equation), so we must use an iterative optimization algorithm like Gradient Descent.



The END

